

Slurm at LIA

This document aims at providing all the necessary information to properly run experiments on the LIA shared cluster, using Slurm. It also describes some good practices that everybody should follow in order to ensure a good experience for all the users of the cluster.

Table of contents

- Running experiments
 - SLURM
 - Submitting a job
 - Getting an interactive session
 - Tracking your running jobs
 - Array of jobs for hyperparameters search
- The LIA SLURM policy
- Summary

Running experiments

At LIA, SLURM is used to ensure a fair amount of compute resources to all the potential users of the cluster. **Eventually, all the experiments relying on the cluster will have to be run via Slurm.**

What does Slurm bring to the lab?

- A fixed policy to ensure the correct use of our shared resources.
- A simple way to start your experiments. Indeed, you will no longer need to look for an available machine as Slurm will do it for you.
- A simple tool to plan your experiments. You can schedule, delay, add e-mail triggers, select specific resources and apply more features to run your experiments.

Slurm

Slurm is an open source, fault-tolerant, and highly scalable cluster management and job scheduling system. First, it is used to attribute resources to users based on their requests. Then, it provides a framework for starting, executing, and monitoring the different jobs. Finally, it arbitrates contention for resources by managing a queue of pending work. More information can be found [here](#).

Submitting a job

The traditional way of running an experiment via Slurm is via the **submission of a job**. Such a process may easily be summarised as follows:

- Create a shell script describing how to run your experiment.
- Ask Slurm to run this script.

Please note that an interactive session can also be requested for debugging purposes. See this section.

In practice, the first point is the hardest to get used to, so let's dive into it. While you can easily ask Slurm to run a single command (e.g think of `http` or `nvidia-smi`), it is expected that your experiment might be a little bit more complex to describe. Hence, it is possible to formalise your experiment into a script so it can easily be started by Slurm. Let's create a first simplified script named `my_experiment.sh`:

```
#!/bin/bash

#
# Other stuff will come here later
#

echo "My first sbatch script!"

python my_super_experiment.py $1 $2
./run_my_super_experiment.sh $1 $2
```

Once this is done, you can ask Slurm to schedule your job with two commands:

- `sbatch <some params> my_experiment.sh` asks the scheduler to run your job as a *detached* job with respect to your current session (with automatic redirection of the standard and error outputs to files).
- `srun <some params> my_experiment.sh` is an *interactive* call, so the output will be attached to your current terminal (and the job will be terminated when the terminal is closed or the connection is lost).

If you find yourself wondering which one to use, the rule is simple: `sbatch` **must always be preferred** over `srun` to schedule your experiments.

With `sbatch`, your script `my_experiment.sh` will be scheduled and will start as an independent process which will continue to run after you log out.

Usage of interactive sessions started with `srun` must be limited to the purpose of debugging (or running limited tests the check the behavior of your software on a small dataset before launching a long job).

!! In particular, never use an interactive session in order to hold on to resources and launch several experiments in a row. Doing so goes against the principle of faire sharing of resources and can lead to your account being suspended.

How to specify the resources needed for my experiment

Resource management is at the core of Slurm. **All Scheduled experiments should specify as precisely as possible the resources needed.**

Resources include: GPUs, CPU cores, RAM, and the expected running time. Such specifications may either be given as arguments to `srun` and `sbatch` or integrated into the experiment script.

Let's schedule our script for 2 hours, 4 cpu cores, and 1 GPU:

```
sbatch -c 4 -p gpu --gpus-per-node=1 --mem=8G --time=2:00:00 my_experiment.sh mysuperparams1 mysuperparams2
```

The same applies for `srun`:

```
srun -c 4 -p gpu --gpus-per-node=1 --mem=8G --time=2:00:00 my_experiment.sh mysuperparams1 mysuperparams2
```

Here `-c` sets the number of CPU cores per task, `-p` specifies the partition to run on (see the next section for more information on partitions), `--gpus-per-node` specifies the number of GPUs, `--mem` is the maximum amount of RAM, and `--time` is the time limit. Please note that **A LOT** of parameters can be passed to these commands. A good summary can be found in this table. For example, it is feasible to ask for an e-mail once your job finishes or crashes with `--mail-type` (you do not need to specify your email address, which is already set for each user account on the cluster).

A good practice, however, is to define these resources directly in your experiment script, using the tag `#SBATCH`. Therefore, if we consider the same setup:

```
#!/bin/bash

#SBATCH --job-name=ThisIsMySuperExperiment
#SBATCH --tasks=1
#SBATCH --cpus-per-task=4
#SBATCH --partition=gpu
#SBATCH --gpus-per-node=1
#SBATCH --mem=8G
#SBATCH --time=2:00:00

echo "My first sbatch script!"

python my_super_experiment.py $1 $2
./run_my_super_experiment.sh $1 $2
```

You can then schedule it with:

```
sbatch my_experiment.sh mysuperparams1 mysuperparams2
```

Resource specifications included in the script (using the `#SBATCH` tag) can be overridden using the command-line options:

```
sbatch --mem=64GB my_experiment.sh mysuperparams1 mysuperparams2
```

Partitions and requesting a specific resource (Tesla P100, 2080 Ti ...)

In the previous example, we asked for a random GPU. Indeed, `--gpus-per-node=1` asks Slurm to find any available GPU. To ask for a specific device, it is necessary to specify the whole resource name. For example, if you want to request an RTX 2080 Ti:

```
sbatch -c 4 -p gpu --gpus-per-node=rtx_2080_ti:1 --mem=8G --time=2:00:00 my_experiment.sh
```

 or a Tesla P100:

```
sbatch -c 4 -p gpu --gpus-per-node=tesla_p100:1 --mem=8G --time=2:00:00 my_experiment.sh
```

A list of the identifiers used for the GPUs in our cluster can be obtained with:

```
sinfo -o '%30G %N'
```

The machines in the cluster are grouped based on the set of features they offer. Such groups are named *partition* by Slurm. The partition on which to run a job can be specified, as seen in the examples above, with option `-p` or `--partition`.

The current partitions are described in the table below:

Name	Description
cpuonly	For jobs which do not use GPUs
gpu	For GPU-powered jobs

If no partition is specified for a job, it will be scheduled on the `cpuonly` partition.

It is also possible to request specific nodes (using their name, like `galactica`) with `--nodelist=NODENAME`.

For more details, you may refer to the official documentation for `srun`: <https://slurm.schedmd.com/srun.html> and `sbatch`: <https://slurm.schedmd.com/sbatch.html>.

Effects of resource specifications

The requested amounts of RAM, CPU cores, and GPUs are of course used by Slurm to find the right machine to run your job.

CPU and GPU

While your job runs, it will be allowed access only to the CPUs and GPUs assigned to it by Slurm.

A multi-GPU job can be written to use all the devices available, since the `nvml` library will only show them the devices assigned to the job.

Threads in a multi-threaded CPU job will be handled the same way as if running on a machine equipped with this number of CPU cores: the Linux scheduler will share all the available cores among all the threads. Thus, launching more threads than the requested number of cores is not blocking. In fact, it is recommended if each of your threads will not consume 100% of a CPU core over all of its lifetime, which is often the case.

✅ Hence, requesting a bit less CPU cores than there will be threads in a job ensures better use of shared resources.

RAM

If a job tries to allocate more RAM than was specified, it will be killed by Slurm. Therefore you must estimate the RAM needs very carefully.

❌ It may be tempting to avoid this risk by largely overestimating the needs, but you are strongly encouraged not to do it, as doing so would be detrimental to all the other users, and may also result in a much longer delay before your job can start running (because of waiting for a large amount of RAM to be available).

Time

A job will also be killed automatically if it runs longer than the time limit specified in the job definition. So, as is the case for RAM, be careful to correctly estimate the time that your job needs to run.

It is safer to slightly overestimate the time limit for your job, to account for possible unexpected slowdowns. One may see the time limit as a safety belt that guarantees that a job will eventually get killed if something goes wrong and the job runs for a lot longer than it should, without you having to check it constantly.

❌ Even though the effect on the cluster of a largely overestimated time limit will not be as strongly detrimental as in the case of RAM, it will still be negative: job scheduling will be less efficient, and other users will not be able to have a correct estimation of when their own jobs will start running. For this reason, you must avoid setting a meaninglessly high time limit for your jobs.

Please note that in a future version of the policy, jobs with long time limits will be more likely to get interrupted to make room for shorter jobs. In the meantime, they are more likely to get manually canceled by the system administrator in cases of low resource availability.

What happens if I do not specify any resource?

By default, all jobs are attached to the `cpuonly` partition. Therefore, you will not be able to use GPUs.

Also, the job will be assigned the default number of cores, amount of RAM, and time limit. It may be sufficient for a small job (e.g. for a quick test), but these default values are deliberately kept low in order to force users to specify resource constraints that really match their jobs.

Getting an interactive session

Sometimes, researchers need to debug their experiment before scheduling multiple runs. To facilitate this debugging, the LIA cluster allows for **short** interactive sessions. An interactive session can be seen as being logged into the corresponding server with the requested resources, so you have an access to a proper prompt and can tweak your experiment before starting a job.

INTERACTIVE SESSIONS CAN NOT EXCEED 1 HOUR. Such a limit is to prevent users from forgetting that an interactive session is running somewhere.

Use `srun` with the option `--pty bash` to request an interactive session. For example, an interactive session with a P100 can be obtained with:

```
srun -c 4 -p gpu --gpus-per-node=1 --mem=8G --time=1:00:00 --pty bash -l
```

During the session, `nvidia-smi` will only display one GPU, and while you can see all the cores of the server with `http`, you will not be able to access more than `4` cores.

Note: It's important to specify option `-l` for the shell (here, `bash`) in order to launch it as a login shell and have the various initialization scripts run to setup your usual environment (particularly for Anaconda, which declares the command `conda` as a shell alias in its initialization script).

Tracking your running jobs

The most important command to track the current state of all scheduled and running Slurm jobs is `squeue`. Therefore, once your job has been scheduled with `sbatch` or `srun` you should be able to see it with `squeue`: Example:

```
(base) [Tue Sep 22 11:28:31 tparcollet@orion ~]$ squeue
JOBID PARTITION NAME USER ST TIME NODES NODELIST(REASON)
90 gpu test-sba tparcoll PD 0:00 1 (None)
82 gpu bash merlin R 16:38 1 talos
83 gpu test-sba tparcoll R 0:05 1 achille
84 gpu test-sba tparcoll R 0:02 1 achille
85 gpu test-sba tparcoll R 0:02 1 atlantia
86 gpu test-sba tparcoll R 0:02 1 aura
87 gpu test-sba tparcoll R 0:02 1 aura
88 gpu test-sba tparcoll R 0:02 1 aura
89 gpu test-sba tparcoll R 0:02 1 galactica
```

If a problem arises with your request (i.e. resources not available), you should see a message in the `NODELIST` column.

Finally, you can ask `squeue` to only show you jobs that are linked to your account with:

```
squeue -u $USER
```

Note that if your job has been started with `srun` you can easily attach it to your terminal at any time with:

```
jobs! JOBID.TASKNUMBER
```

 with `JOBID` corresponding to the `JOBID` column of `squeue`. This is not feasible with an `sbatch` job!

In the case of a `sbatch` job, the contents of the standard output can be found in the automatically generated Slurm log file. By default, Slurm produces a log file named `slurm-JOBID.out` everytime a `sbatch` job starts to run. This setup can be changed with `--output=mysuperpath/mysuperfile.out`.

The official documentation for `squeue`.

Display job details

If you want to see details about a specific job, such as its run time, the nodes it is (or was) running on, the reason why it is not running yet, or the reason why it was killed, you can use:

```
scontrol show job <jobID>
```

How to kill my jobs

Your jobs can be killed with `scancel JOBID`. If you want to kill **all** your scheduled and running jobs you can use

```
scancel -u $USER.
```

How to automatically reschedule a job that was killed

If one of your jobs disappears from `squeue`, it is likely that it got killed in order to make room for a higher priority job. To allow such a job to be instantaneously rescheduled, you can use the `SBATCH --requeue` option.

Array of jobs for hyperparameters search

Slurm can help you handle the hyperparameters tuning phase of your experiments. More precisely, Slurm can manage an *array* of jobs for you, with each job of this array being a different call.

Let's consider a straightforward investigation of the number of layers in a deep neural network. To achieve this goal, we just need to customize our `my_experiment.sh` script. We can make use of some of the environment variables defined by Slurm:

- `SLURM_ARRAY_JOB_ID`: same value as `SLURM_JOB_ID` → the job ID for the whole array.
- `SLURM_ARRAY_TASK_ID`: the index in the array for each sub-job.
- `SLURM_ARRAY_TASK_COUNT`: the number of sub-jobs in the job array.
- `SLURM_ARRAY_TASK_MAX`: the highest index value in the job array.
- `SLURM_ARRAY_TASK_MIN`: the lowest index value in the job array.

```
#!/bin/bash

#SBATCH --job-name=ThisIsMySuperExperiment
#SBATCH --tasks=1
#SBATCH --partition=gpu
#SBATCH --cpus-per-task=4
#SBATCH --gpus-per-node=2
#SBATCH --mem=8G
#SBATCH --time=4:00:00
#SBATCH --array=0-3

# Activate the conda env if needed
source /etc/profile.d/conda.sh # Required before using conda
conda activate myenv

LAYERS=(25 50 100 200)

srun python train.py --bits ${LAYERS[${SLURM_ARRAY_TASK_ID}]}
```

An advantage of this approach is that *arrays* of jobs do not appear in `squeue` if jobs aren't allocated. More precisely, you will only see the main array job and the ones that are currently running. This way, if you have thousands of super short jobs to launch, you won't be responsible for a massive flood of the scheduler. An example of `squeue` with an array of job is:

```
95_[5-10] gpu ThisIsMy tparcoll PD 0:00 1 (Resources)
95_0 gpu ThisIsMy tparcoll R 0:02 1 talos
95_1 gpu ThisIsMy tparcoll R 0:02 1 talos
95_2 gpu ThisIsMy tparcoll R 0:02 1 talos
95_3 gpu ThisIsMy tparcoll R 0:02 1 talos
95_4 gpu ThisIsMy tparcoll R 0:02 1 talos
```

Here we can see that from an array of 10 jobs ([5-10]), only the first 5 are running.

!! When your array is larger than 4 or 5 jobs, add a `%n` specifier at the end of the array definition to limit the number of jobs of the array that can run simultaneously:

```
#SBATCH --array=1-15%4
```

In the example above, 15 jobs will be scheduled, but at most 4 will be run at the same time.

The LIA Slurm policy

The policy will be detailed after the testing period.

Summary

- Policy will be detailed later.
- Schedule your jobs with `sbatch` by writing an experiment script.
- Use interactive sessions (started with `srun`) for debugging.
- Use `squeue`, `sattach` and `scancel` to track your running and scheduled jobs.
- Important commands and options: http://www.physik.uni-leipzig.de/wiki/files/slurm_summary.pdf.